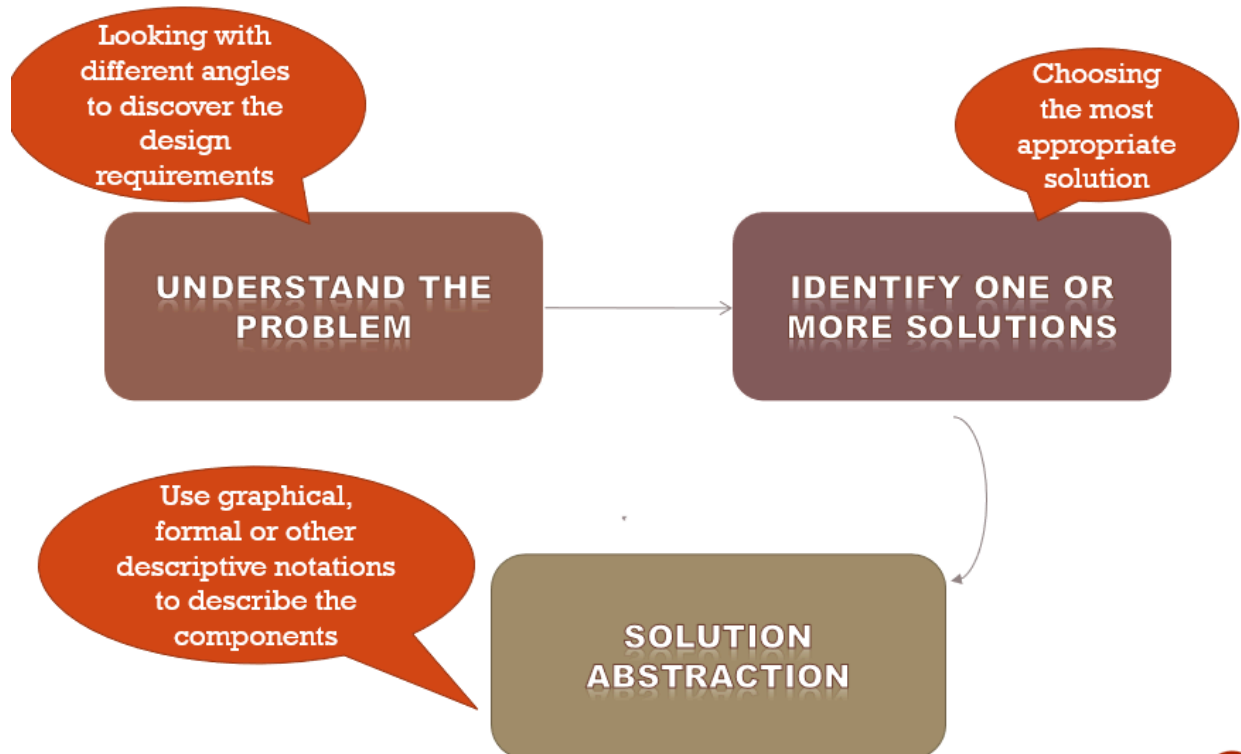


Design process

STAGES OF DESIGN



GOOD DESIGN VS BAD DESIGN

Bad Design	Good Design
A design that is vague about underlying requirements	Good design is the one in which all the customer's requirements are incorporated
When you make a change in one piece of code and it breaks some other part which you never expected	Good designs are ones that are easily readable, easily maintainable, easily understood (good interface)
Bad user interface would add pain and complexity for user	A good software has the right amount of separation of concerns and modularity (the code encapsulated in the right logical tiers/modules/layers) making it maintainable.
implicit requirements are unmentioned, for example ease of use, maintainability etc	

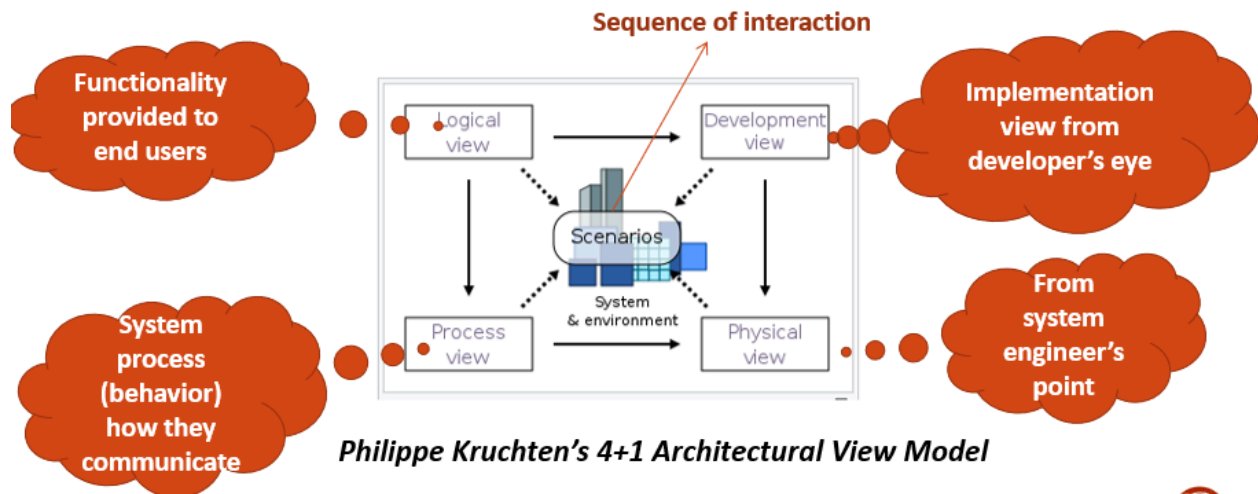
"a product's quality is a function of how much it changes the world for the better." – meaning user satisfaction is more important than anything in determining software quality.

DESIGN CONCEPTS

1. Abstraction
2. Architecture
3. Design Patterns
4. Modularity
5. Information Hiding
6. Functional Independence
7. Refinement
8. Refactoring
9. OO design concepts
10. Design Classes—provide design detail that will enable analysis classes to be implemented

CHARACTERISTICS OF ARCHITECTURE DESIGN

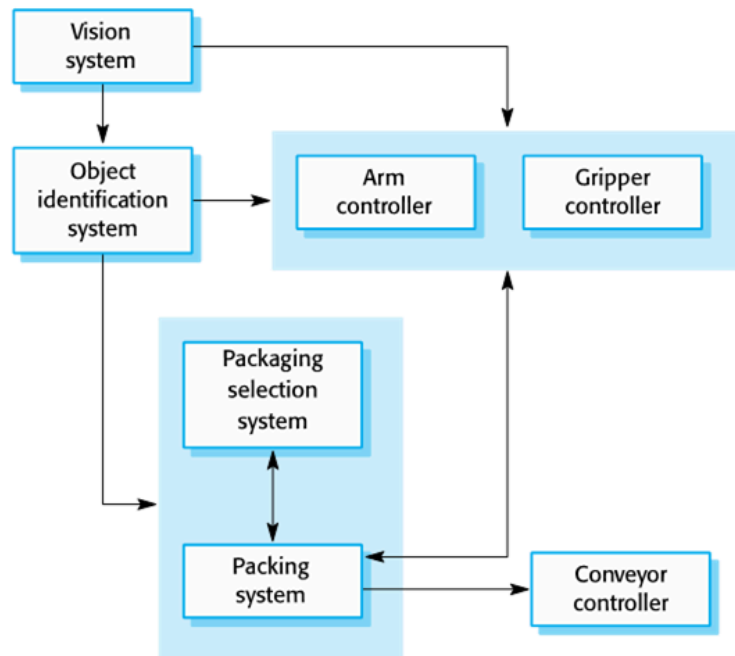
- ✓ multiple views of stakeholders
- ✓ In order to reduce the complexity **separate design model view points** for various stakeholders should be provided – *separation of concern*



- Architectural patterns
 - Define the overall structure of software
 - Indicate the relationships among subsystems and software components
 - Define the rules for specifying relationships among software elements
- Design patterns
 - Address a specific element of the design such as an aggregation of components or solve some design problem, relationships among components, or the mechanisms for effecting inter-component communication
 - Consist of creational, structural, and behavioral patterns
- Coding patterns
 - Describe language-specific patterns that implement an algorithmic or data structure element of a component, a specific interface protocol, or a mechanism for communication among components

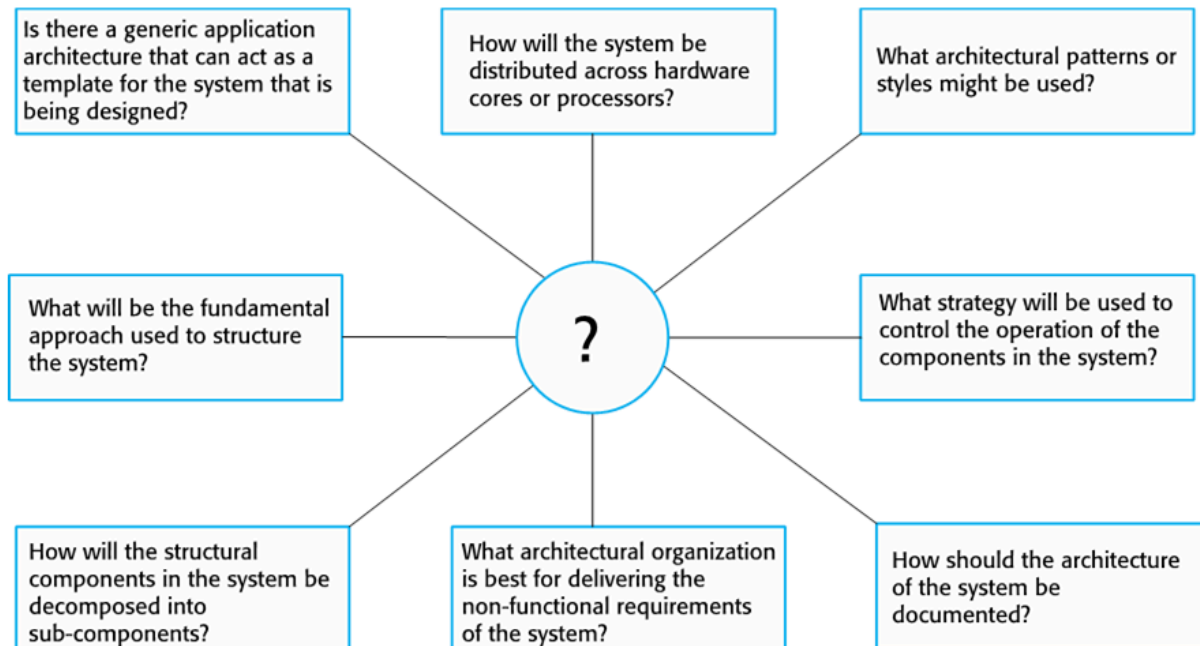
BOX AND LINE DIAGRAMS

- Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- However, useful for communication with stakeholders and for project planning.



The Architecture of a Packing Robot Control System

ARCHITECTURAL DESIGN DECISIONS

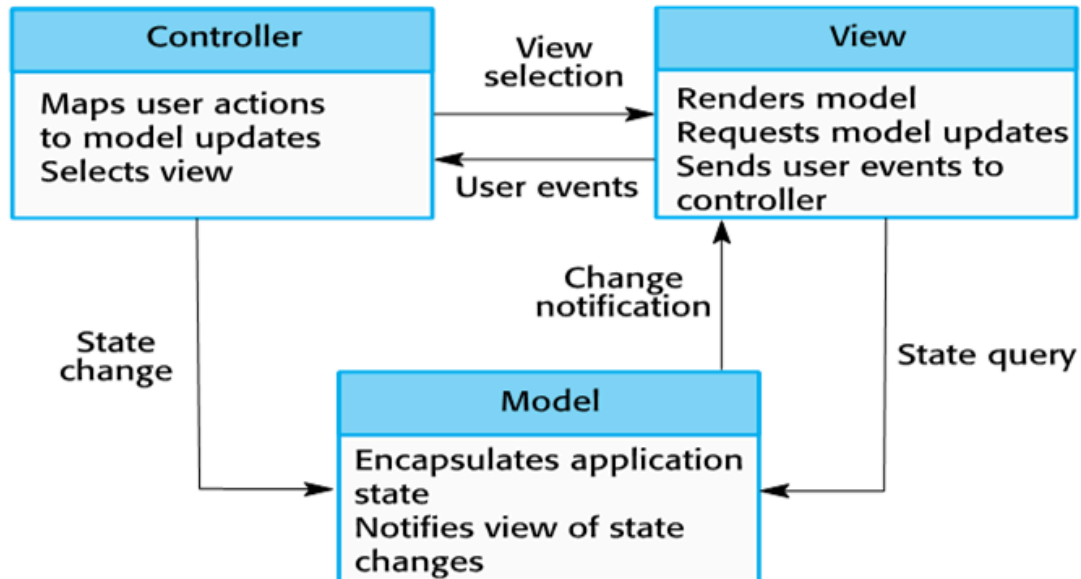


39

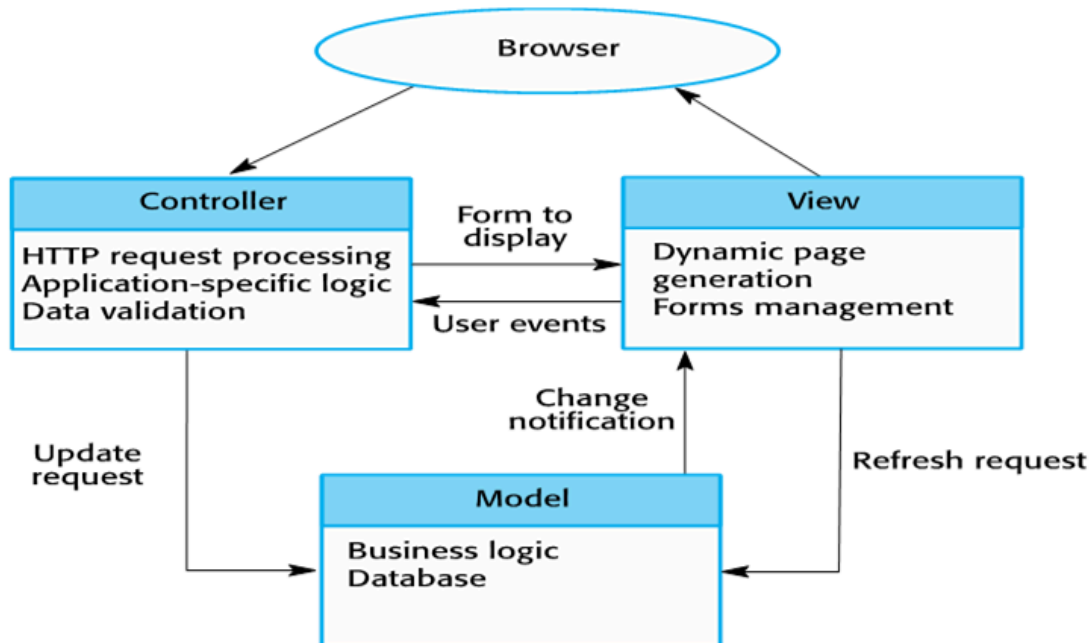
THE MODEL-VIEW-CONTROLLER (MVC) PATTERN

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.3.
Example	Figure 6.4 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

THE ORGANIZATION OF THE MODEL-VIEW-CONTROLLER



WEB APPLICATION ARCHITECTURE USING THE MVC PATTERN

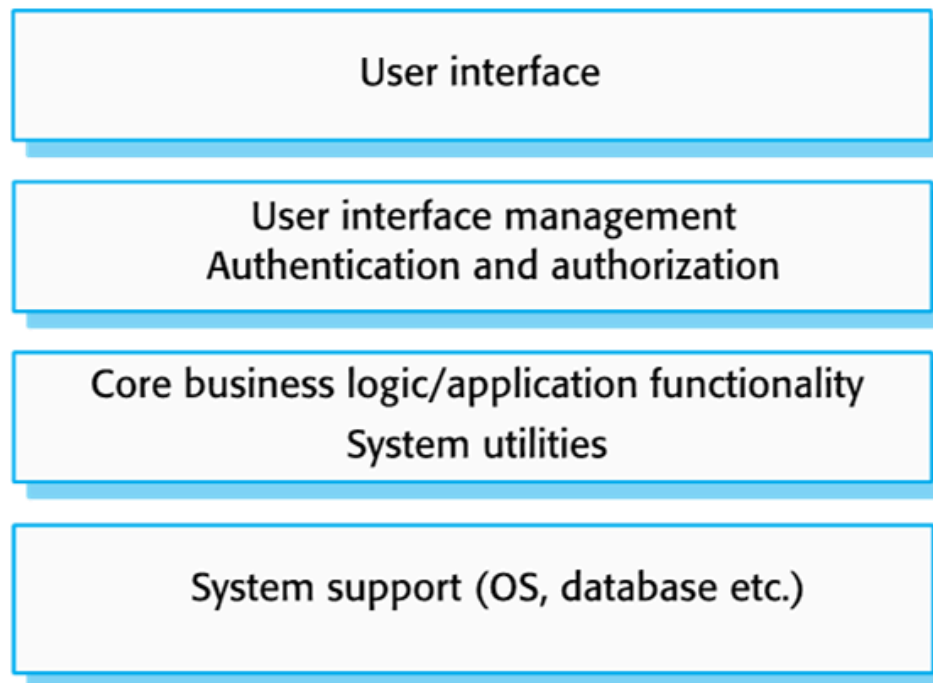


THE LAYERED ARCHITECTURE PATTERN

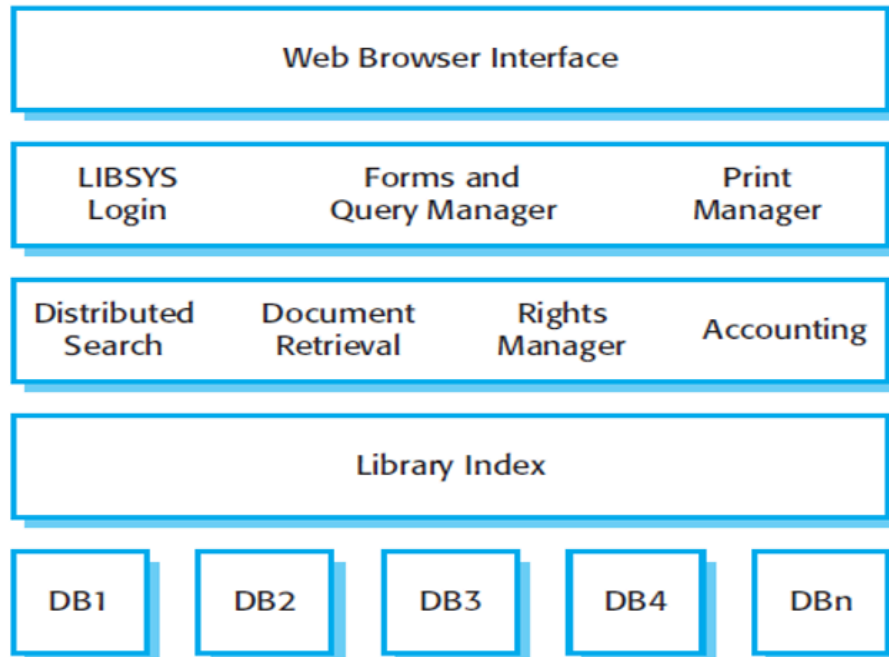
Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.



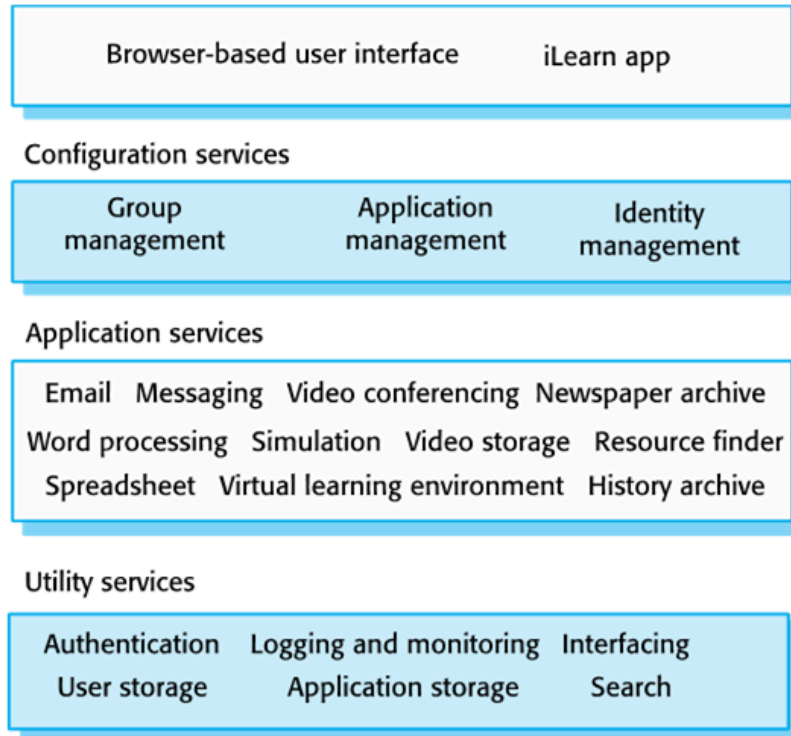
A GENERIC LAYERED ARCHITECTURE



THE ARCHITECTURE OF THE LIBSYS SYSTEM



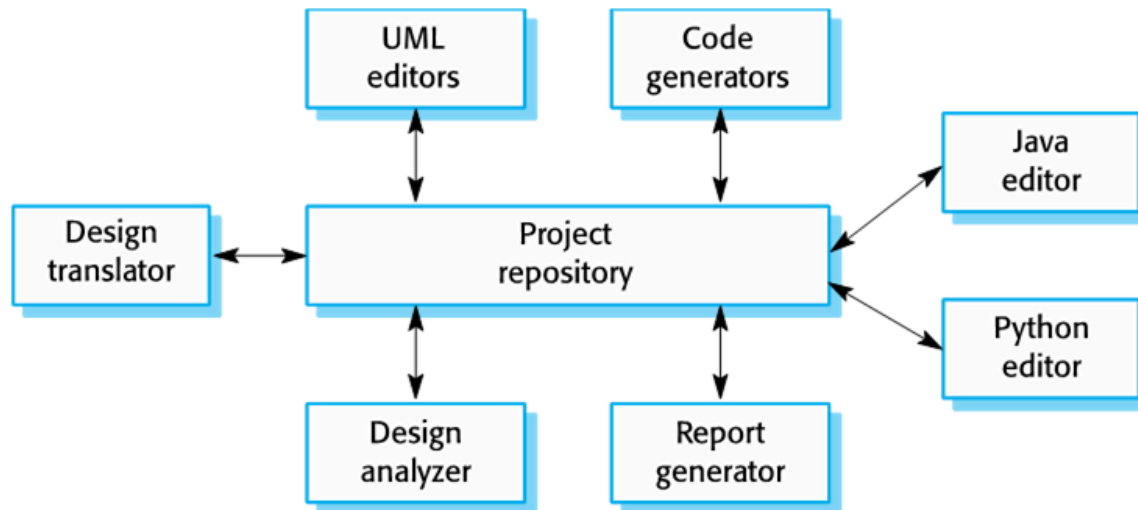
THE ARCHITECTURE OF THE ILEARN SYSTEM



THE REPOSITORY PATTERN

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

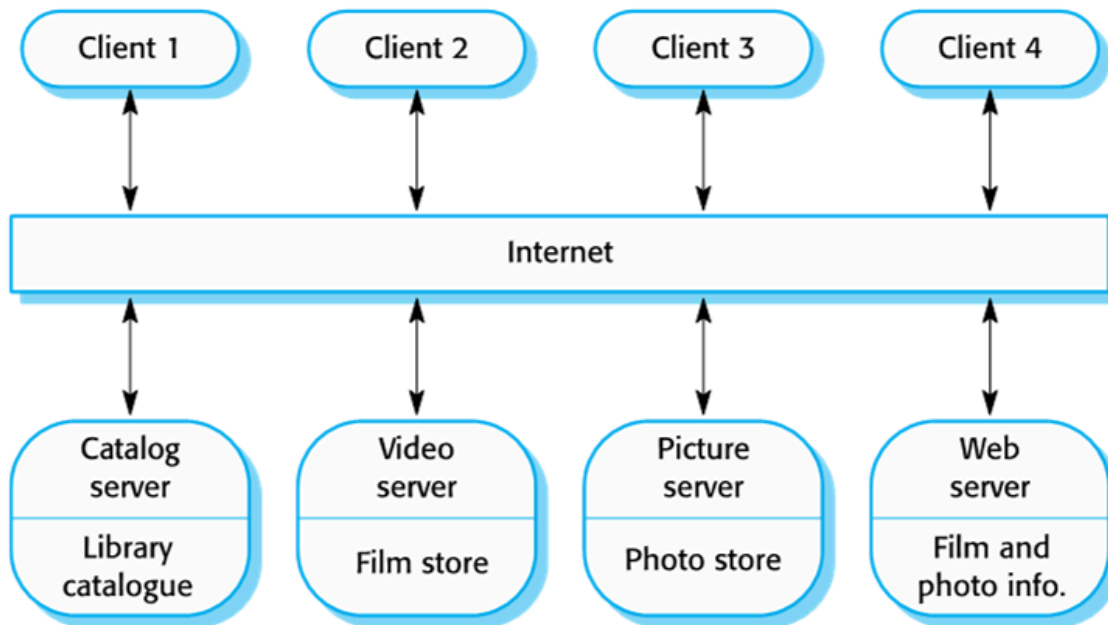
A REPOSITORY ARCHITECTURE FOR AN IDE



THE CLIENT–SERVER PATTERN

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.11 is an example of a film and video/DVD library organized as a client–server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

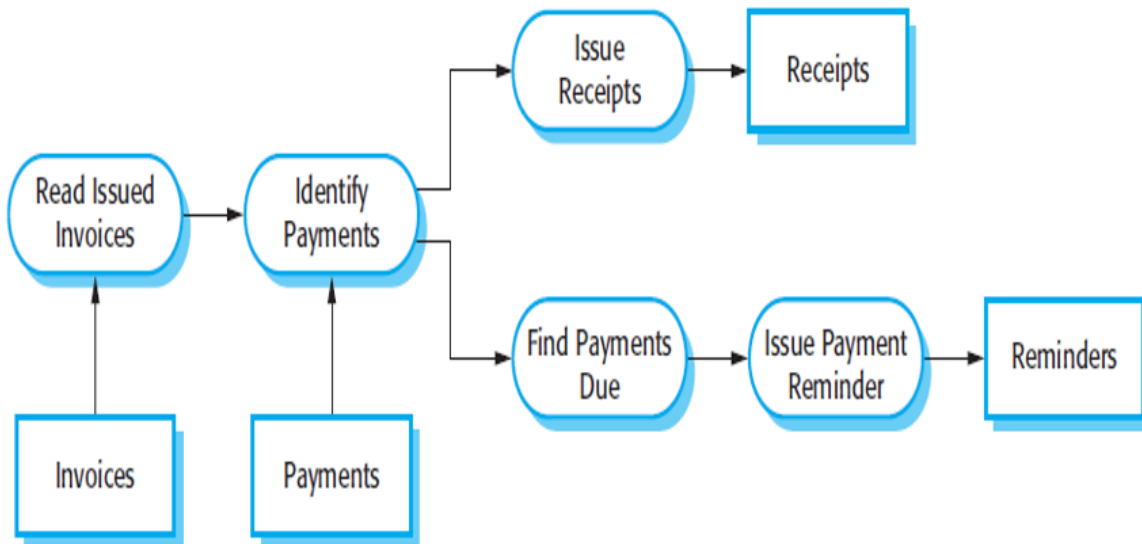
A CLIENT–SERVER ARCHITECTURE FOR A FILM LIBRARY



THE PIPE AND FILTER PATTERN

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.13 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

AN EXAMPLE OF THE PIPE AND FILTER ARCHITECTURE USED IN A PAYMENTS SYSTEM



✓ APPLICATION ARCHITECTURES

- Designed to meet **organizational/business needs**
 - businesses w similar needs → have **similar architectures**
 - These common structures are called **generic application architectures**
 - A **generic architecture** can be:
 - **Reused**
 - **Modified**
 - **Adapted** for specific system requirements
-



USE OF APPLICATION ARCHITECTURES

- ✓ Starting point for architectural design
 - ✓ Design checklist
 - ✓ Organizing development work
 - Helps divide tasks among team members
 - ✓ Assessing reuse
 - ✓ Common vocabulary
-



EXAMPLES OF APPLICATION TYPES

♦ 1. Data Processing Applications

- Process large amounts of data in **batches**
 - **No user interaction** during **processing**
-

♦ 2. Transaction Processing Applications

- Handle **user requests**
 - **Update** and **manage database information**
-

♦ 3. Event Processing Systems

- System reacts to **events from environment**
 - **Actions depend on detected events**
-

♦ 4. Language Processing Systems

- **Users give** input in a **formal language**
 - **System interprets** and **executes** it
-



APPLICATION TYPE EXAMPLES

♦ Transaction Processing Systems

- **E-commerce systems**
 - **Reservation systems**
-

♦ Language Processing Systems

- **Compilers**
 - **Command interpreters**
-



TRANSACTION PROCESSING SYSTEMS

- Handle **user requests** for:
 - **Retrieving data**
 - **Updating database**
-

♦ Transaction (from user perspective):

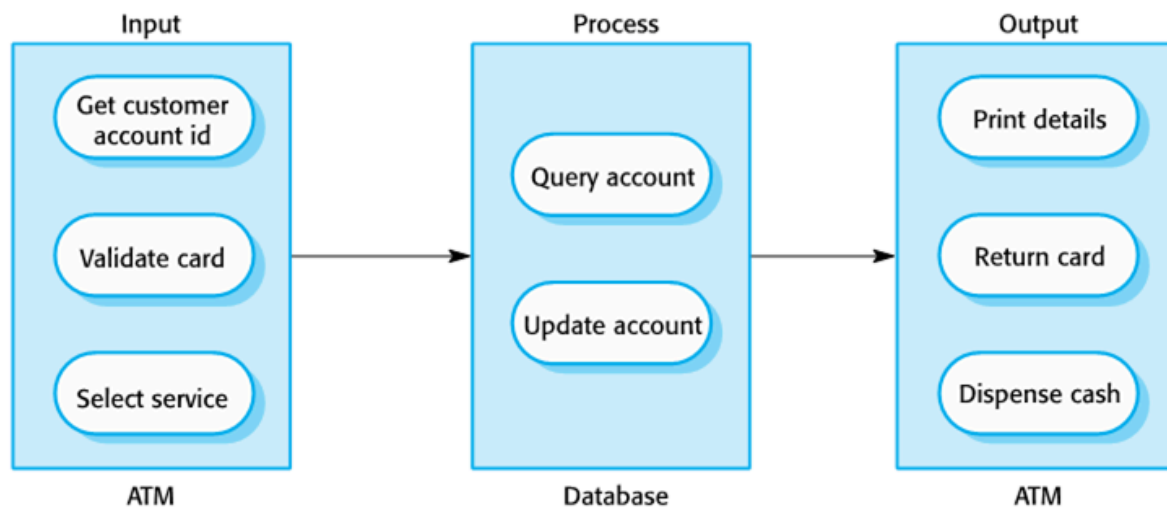
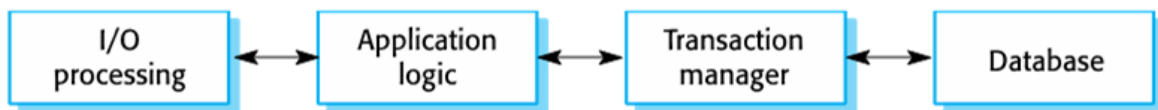
- A **sequence of operations** to **achieve a goal**

 Example:

- Searching flight timings from one city to another

♦ **Working:**

- Users send **requests asynchronously** (at different times)
- Requests are handled by a **transaction manager**



✓ INFORMATION SYSTEMS ARCHITECTURE

- Have **generic layered architecture**
- Organized into **different layers**, each with a specific role

- These systems are mostly **transaction-based**
 - Users interact by performing **database transactions**
-

LAYERS IN INFORMATION SYSTEM ARCHITECTURE

- 1) **User interface**
- 2) **User communications**
- 3) **Information retrieval**
- 4) **System database**

♦ **1. User Interface Layer**

- The part users interact with
 - **Displays information and takes input**
 - Example: forms, dashboards, web pages
-

♦ **2. User Communications Layer**

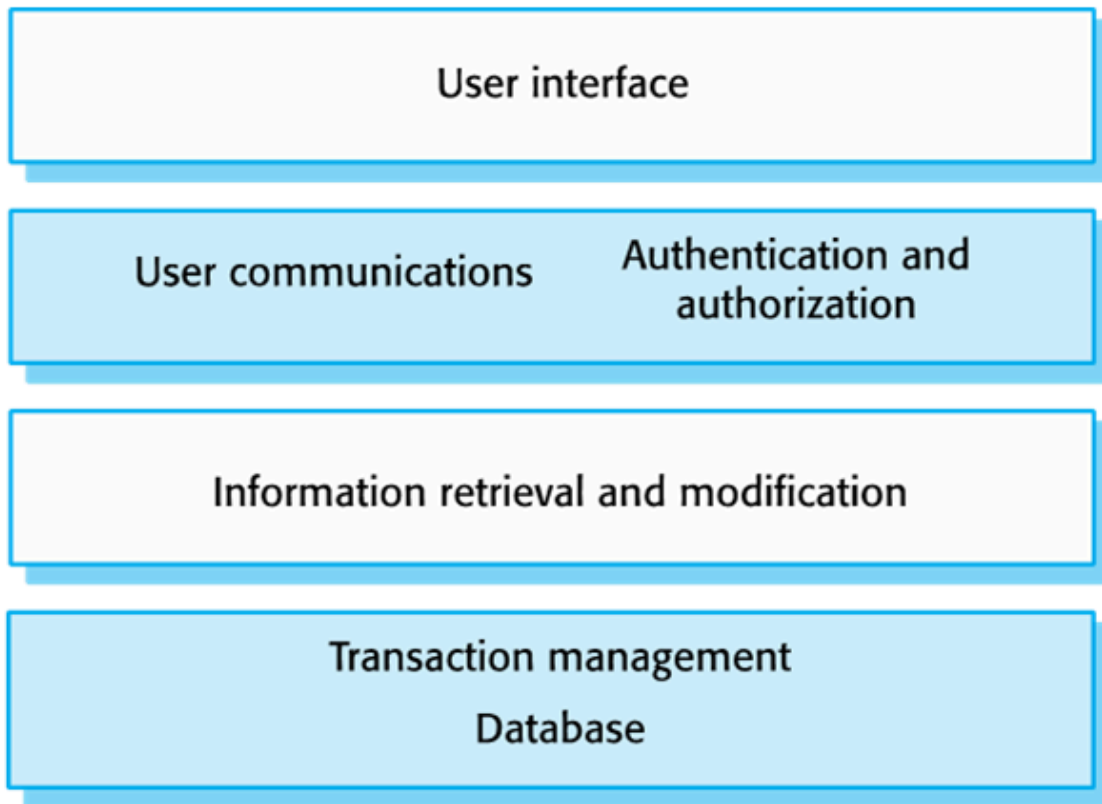
- Handles **communication between user and system**
 - Manages **requests and responses**
 - Example: sending user input to the system and returning results
-

♦ **3. Information Retrieval Layer**

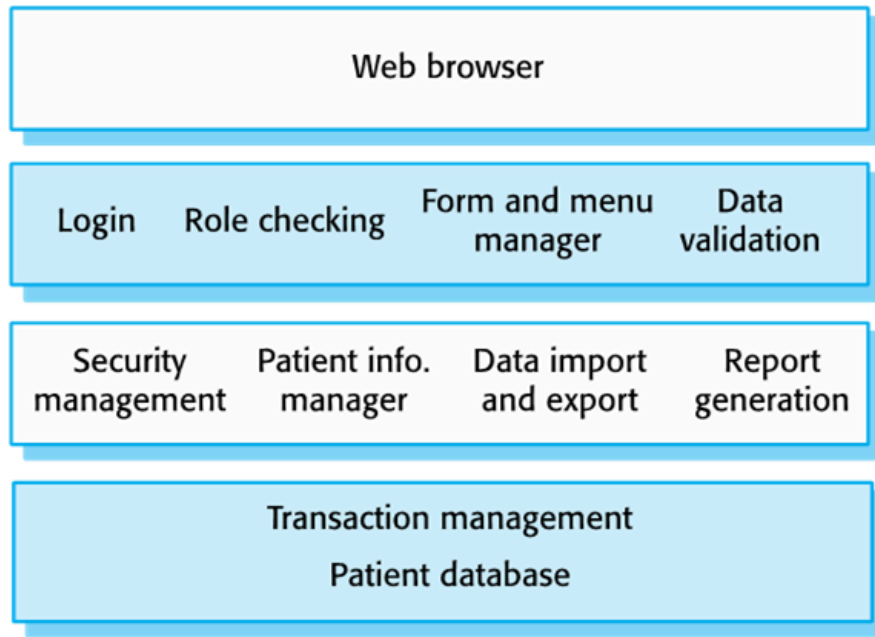
- **Processes user requests**
- **Fetches** and **manipulates** required **data**
- Applies **business logic**

◆ 4. System Database Layer

- **Stores all system data**
- Performs **database operations** (insert, update, delete, retrieve)



THE ARCHITECTURE OF THE MENTCARE SYSTEM



WEB-BASED INFORMATION SYSTEMS

- Modern information systems are mostly **web-based**
- Users interact through a **web browser (UI layer)**
- Used for **information and resource management**

♦ Example: E-commerce Systems

- **Internet-based systems for buying/selling goods**
- Users can:
 - Place orders
 - Make payments
 - Get delivery

♦ Shopping Cart Functionality

- Users can add items in **multiple steps (transactions)**
 - All items are purchased together in **one final transaction**
 - Provides **flexibility and convenience**
-

SERVER IMPLEMENTATION (Multi-tier Architecture)

These systems follow a **multi-tier client-server architecture**

♦ 1. Web Server

- Handles **user communication**
 - User Interface ⇒ **web browser**
 - Displays UI and receives user input
-

♦ 2. Application Server

- Contains **business logic**
 - **Processes user requests**
 - Handles:
 - **Data processing**
 - **Information retrieval**
-

♦ 3. Database Server

- **Manages the database**
 - Performs:
 - **Data storage**
 - **Data retrieval**
 - **Transaction management**
-



LANGUAGE PROCESSING SYSTEMS

- Accept **input** in a **natural or formal language**
 - Convert it into **another representation or output**
-

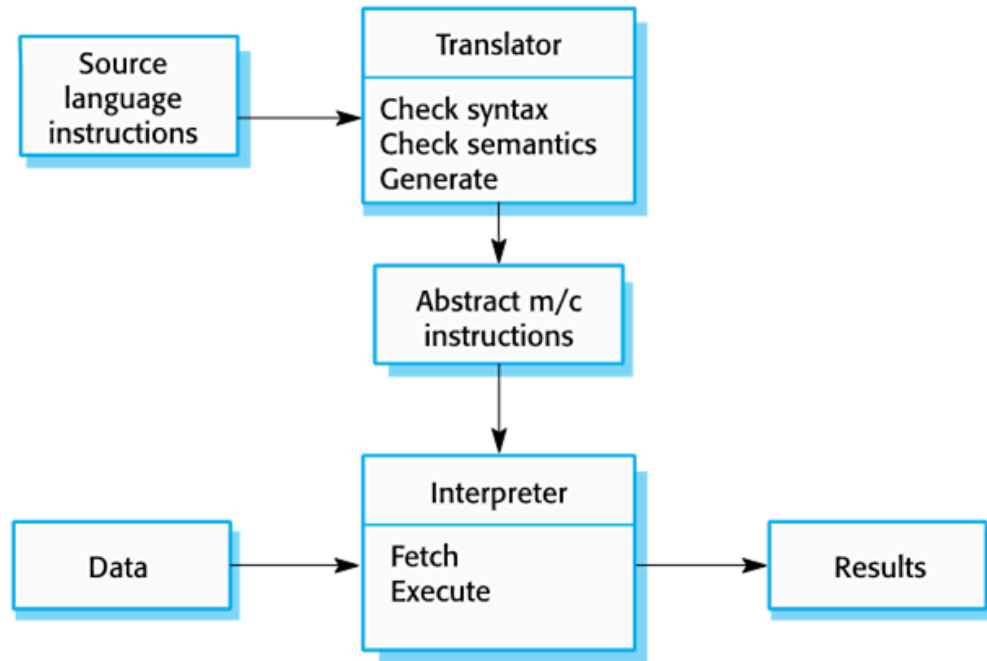
♦ Key Features

- May include an **interpreter**
 - **Executes instructions written in the language**
 - Used when problems are best solved by:
 - Writing algorithms
 - Describing system data
-

♦ Examples

- Compilers
- Command interpreters

THE ARCHITECTURE OF A LANGUAGE PROCESSING SYSTEM



✓ COMPILER COMPONENTS

Flow : **Lexical** → **Syntax** → **Semantic** → **Code Generation**

♦ 1. Lexical Analyzer (Scanner)

- Takes **input source code**
 - Breaks it into **tokens** (keywords, identifiers, symbols)
 - Converts input into an **internal format**
-

♦ 2. Symbol Table

- Stores information about **identifiers**
 - **Variables**
 - **Class names**
 - **Object names**
 - Used throughout compilation for **reference and validation**
-

♦ 3. Syntax Analyzer (Parser)

- Checks if code follows **grammar rules**
 - **Detects syntax errors**
 - Builds structure of the program
-

♦ 4. Syntax Tree

- **Internal representation** of the program
 - Shows **hierarchical structure of code**
 - Used by later stages of compilation
-

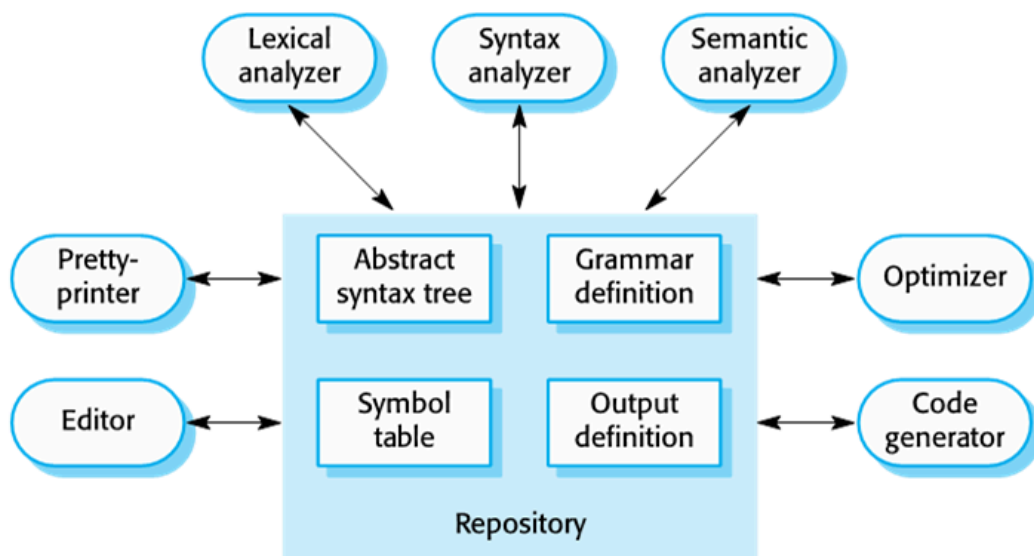
♦ 5. Semantic Analyzer

- Checks **meaning (semantics)** of the program
 - Uses:
 - Syntax tree
 - Symbol table
 - Detects errors like:
 - **Type mismatch**
 - **Undeclared variables**
-

♦ 6. Code Generator

- Traverses (walks) the syntax tree
- Generates **target code / machine code**
- Final stage of compilation

A REPOSITORY ARCHITECTURE FOR A LANGUAGE PROCESSING SYSTEM



A PIPE AND FILTER COMPILER ARCHITECTURE

